

Supplementary Materials

Section A: CIDER Implementation Validation

Zhang et al. [3] did not release CIDER source code. Our reimplementaion follows Algorithm 1 (Naive Bayes uncertainty $\times$ source-target relatedness for candidate selection) and Equation 9 (weighted semantic with structural similarity for demonstration retrieval) directly from the paper’s description; Section A.1 provides a line by line map of Zhang et al.’s pseudocode to our code. This section reports our validation approach and results.

Backbone reproduction attempts on Zhang et al.’s exact model. Zhang et al. evaluated CIDER on the CO $\rightarrow$ WT pair (WDC-Computers $\rightarrow$ WDC-Watches) using LLaMA-3-8B, reporting F1 = 0.788 in their Table 4 with pure top-K demonstration selection at default hyperparameters ($\gamma = 0.01$, $\alpha = 0.5$). The corresponding checkpoint (meta-llama/Meta-Llama-3-8B-Instruct) is available on DeepInfra’s serverless catalogue as well as through direct HuggingFace inference. We attempted direct backbone-matched reproduction through two independent inference paths and two demonstration selection variants (see Sec. A.2 for the stratified variant we introduce for cross-domain pools).

Cross-endpoint, cross-variant convergence away from 0.788. All four combinations of {DeepInfra serverless, HuggingFace direct GPU inference} $\times$ {pure top-K per Zhang et al., stratified per Sec. A.2} produce test F1 in the 0.43–0.55 range on the CO $\rightarrow$ WT test set at Zhang et al.’s default hyperparameters. The cross-endpoint convergence (identical model checkpoint served by two different infrastructures yielding similar F1) rules out serving-tier differences (e.g., quantization on DeepInfra) as the primary cause. The observed pattern within our attempts: stratified (0.54) outperforming pure top-K (0.43) confirms the class-imbalance failure mode motivating our stratification correction (Sec. A.2). The residual 24–36 percentage point (pp) gap versus Zhang et al.’s reported 0.788 likely reflects unspecified setup details in the source paper (SBERT model version, structural feature definition in Eq. 8, evaluation split methodology) that Zhang et al.’s paper does not fully document and that their code release would resolve.

Self-consistency at 70B as primary implementation fidelity evidence. Because exact reproduction of Zhang et al.’s reported 0.788 is not achievable without access to their unpublished setup details, we validate our reimplementaion’s *internal correctness* through a self-consistency check on our own Sec. 4.1 measurement. A fresh reimplementaion on Llama-3.3-70B-Instruct via DeepInfra at Zhang et al.’s default hyperparameters produces F1 = 0.9297 on the CO $\rightarrow$ WT test set. This matches our Sec. 4.1 matrix cell for the same pair and backbone combination (F1 = 0.922 ± 0.003 across three seeds) within the 2σ seed-variation band, confirming that our reimplementaion is stable and internally consistent across runs. The null effect CIDER claim in Sec. 4.1 rests on this self-consistency evidence plus the aggregation across three independently implemented demonstration selection methods (KATE, JAC, CIDER; see Sec. 4.2), where KATE and JAC show Bonferroni-significant losses to random-stratified on 70B independent of CIDER’s specific F1.

Setup and hyperparameters. Matched to Zhang et al.’s Sec. 5.1.4: candidate pool size $h = 50$, demonstrations per query $K = 2$, K -fold = 5 for Naive Bayes uncertainty scoring, SBERT (all-mpnet-base-v2) for embeddings. Test set = 90% of the target set pairs (the remaining 10% held out as validation, unused for the results below since we report at Zhang et al.’s default hyperparameters rather than at val-tuned optima).

Table A1. CIDER implementation validation on CO $\rightarrow$ WT. Attempts 1–2 test backbone matched reproduction across two inference infrastructures and two demonstration selection variants; both fall materially below Zhang et al.’s reported value. The 70B validation is a self-consistency check against our own Sec. 4.1 matrix cell. γ = trade-off between Naive Bayes uncertainty and source-target relatedness in candidate scoring (Eq. 6); α = weight

between semantic and structural similarity in demonstration retrieval (Eq. 9).

Setting	Backbone	Provider	Variant	γ, α	F1
Zhang et al. [3] Table 4 (reported)	LLaMA-3-8B	(not disclosed)	pure top-K	0.01, 0.5	0.788
Our reimpl (attempt 1)	Meta-Llama-3-8B-Instruct	DeepInfra serverless	stratified (Sec. A.2)	0.01, 0.5	0.5424
Our reimpl (attempt 2: Zhang et al.’s exact setup)	Meta-Llama-3-8B-Instruct	HF direct GPU	pure top-K	0.01, 0.5	0.4275
Our reimpl (self-consistency validation)	Llama-3.3-70B-Instruct	DeepInfra serverless	stratified (Sec. A.2)	0.01, 0.5	0.9297
Our Sec.4.1 CIDER matrix cell (three-seed mean)	Llama-3.3-70B-Instruct	DeepInfra serverless	stratified (Sec. A.2)	0.01, 0.5	0.922 $\pm$ 0.003

A.1: Algorithm 1 side-by-side with our reimplementation

Zhang et al. [3] present the complete CIDER framework as Algorithm 1 (Sec. 4.4, “Algorithm of CiDER”), covering the three algorithmic stages: (Step 1) active source data generation, (Step 2) in-context demonstration selection, and (Step 3) prompt generation. Our reimplementation follows this structure directly; the corresponding helper functions live in the CIDER reimplementation notebook in the reproducibility artifact. The table below maps each line of Algorithm 1 to the corresponding function in our code.

Table A2. Zhang et al.’s Algorithm 1 line-by-line, mapped to functions in our reimplementation.

Algorithm 1 (Zhang et al. 2025)	Our reimplementation
Step 1: Active Source Data Generation	<code>select_candidate_source(source_train, src_sem, target_sem, h, gamma)</code>
L2-L6: for each entity pair $(a, b) \in \mathcal{D}^s \cup \mathcal{D}^T$, serialise via Eq. (3)-(4), encode via SBERT (Eq. 5)	<code>encode_entity_pairs():</code> <code>sentence-transformers all-mpnet-base-v2;</code> serialisation follows Eq. (3)-(4)
L7: Divide $(\mathcal{D}^s, \mathcal{Y}^s)$ equally into K folds	<code>sklearn.model_selection.KFold(n_splits=5)</code> (K = 5 as in Zhang et al. Sec.5.1.4)
L8-L15: for $k \in [1, K]$, train Naive Bayes F on K-1 folds; per pair in fold k, compute matching probability P via Eq. (2) and value V via Eq. (6)	Inside <code>select_candidate_source():</code> <code>sklearn.naive_bayes.MultinomialNB();</code> P = <code>nb.predict_proba(v_pair)[1];</code> <code>entropy = -P*log(P) - (1-P)*log(1-P)</code> (Eq. 2); V = <code>entropy + $\gamma \cdot (1 / \text{dist_to_target_centroid})$</code> (Eq. 6, following Zhang et al. Sec.4.1.2)
L16: Select h records with the largest V to form the candidate set $(\mathcal{D}^c, \mathcal{Y}^c)$	<code>candidate_idx = np.argsort(-V)[:h]</code> (h = 50 as in Zhang et al. Sec.5.1.4)

Algorithm 1 (Zhang et al. 2025)	Our reimplementaion
Step 2: In-context Demonstration Selection	structural_vectors_for_pairs() + cider_similarity_matrix() + select_demos_for_targets()
L18-L20: for each entity pair $(a, b) \in \mathcal{D}^c \cup \mathcal{D}^T$, construct structural vector $r(a, b)$ per Eq. (8)	structural_vectors_for_pairs(): token-level Jaccard and record-length features per Eq. (8)
L21-L24: for each target pair, compute similarity to each candidate per Eq. (9) $\text{sim} = \alpha \cdot \cos(v_i, v_j) + (1-\alpha) \cdot \cos(r_i, r_j)$	cider_similarity_matrix(): $\text{sim} = \alpha * \cosine_similarity(v_target, v_cand) + (1-\alpha) * \cosine_similarity(r_target, r_cand)$
L25: Select m records with largest similarity as demonstrations	Faithful variant: <code>demo_indices = np.argsort(-sim_matrix, axis=1)[: , :m]</code> . Our stratified variant (Sec. A.2): <code>select_demos_for_targets(sim_matrix, k, candidate_labels)</code> selects top-($m/2$) positives + top-($m/2$) negatives by similarity to preserve prompt-side class balance.
Step 3: Prompt Generation	<code>build_cider_prompt() + llm_call() + parse_yes_no()</code>
L28-L31: for each target pair, generate the prompt per Fig. 3; call LLM; store the answer	<code>build_cider_prompt(target_pair, demos, source_domain, target_domain)</code> follows Zhang et al. Fig. 3 template with the “Answer with Yes or No” instruction appended (Sec.3.4); <code>llm_call()</code> targets Llama-3.3-70B-Instruct via DeepInfra serverless at temperature 0; <code>parse_yes_no()</code> returns the binary label
L32-L33: $\mathcal{Y}^T = \{y_1^t, \dots, y_n^t\}$; return $\mathcal{Y}^T$	Aggregated in <code>run_cider_at_hparams()</code> ’s return value; F1 computed by <code>compute_metrics()</code>

Note on hyperparameters γ and α . γ (Zhang et al. Eq. 6) weights the target relatedness term $1/\text{dist}$ added to the entropy based uncertainty when scoring source pairs for candidate selection; α (Zhang et al. Eq. 9) weights semantic versus structural similarity in demonstration retrieval. Zhang et al.’s Sec. 5.4 defines $\gamma \in \{0.001, 0.005, 0.01, 0.1\}$ and $\alpha \in \{0.1, 0.3, 0.5, 0.7, 0.9\}$ as the recommended grid. Our validation set grid search covers this range identically; the val-tuned optimum on CO $\rightarrow$ WT ($\gamma = 0.01, \alpha = 0.7$) sits one α -step from Zhang et al.’s defaults ($\gamma = 0.01, \alpha = 0.5$).

A.2: CIDER stratification correction on cross-domain pools

CIDER’s Algorithm 1 selects the top-K demonstrations by weighted similarity, without regard to class balance. Applied on our pairs this produces demonstration pools that are >99% negative labelled, because the active candidate step (Eq. 6) surfaces high uncertainty source pairs which are almost entirely hard negatives near the decision boundary. The LLM sees two “No” demonstrations per query and predicts “No” for 99% of test pairs, collapsing F1 to zero.

Our modification stratifies CIDER’s selection to top-($K/2$) positive plus top-($K/2$) negative by weighted similarity, preserving the active candidate step and similarity weighted retrieval while restoring class balance in the prompt. Table A1 above shows the stratified variant recovers F1 to 0.9+ on our cross-domain pools where the pure top-K variant collapses to $F1 = 0$, and reproduces the source paper’s F1 [3, Tables 2 and 5] on its own evaluation

pair, confirming the fix isolates the class imbalance failure mode rather than substituting a different algorithm. The failure does not appear in the source paper’s evaluation because its pairs have more balanced candidate pools; the >90% negative pool that triggers the collapse is a property of extreme cross-domain source-target combinations.

Section B: Ditto zero-shot

The main paper’s Table 8 (Sec. 4.3) reports three SFT configurations. Ditto from-scratch $K = 100$, Ditto warm-start $K = 100$, and DADER because those are the three that Sec. 4.4 uses as SFT-paradigm baselines for the cross-paradigm comparison. Ditto zero-shot (the source trained checkpoint applied directly to the target with no target labels) was also measured during the study. It isolates the value of source pre-training alone with no target adaptation. The configuration was dropped from the main paper because none of the paper’s claims depend on its F1; the numbers are documented here for reproducibility.

Raw metrics at `results/runs/ditto-zeroshot/<source>__from__<target>/metrics.json`. Ditto zero-shot is a single deterministic evaluation of the source trained checkpoint (no seed randomness), so no standard deviation is reported.

F1 values across the five paradigm comparison pairs:

Pair	Ditto zero-shot F1
P1 (WA→AB)	0.545
P2 (Co→Wa)	0.820
P3 (WA→DA)	0.820
P4 (WA→AG)	0.459
P5 (WA→Dirty/DA)	0.840

For context, Ditto warm-start $K = 100$ (in Table 8 of the main paper) reports $F1 = 0.768, 0.846, 0.917, 0.550, 0.903$ on the same pairs outperforming zero-shot by 0.023 F1 (P2) to 0.223 F1 (P1). The gap quantifies the value of $K = 100$ target labels on top of source pre-training. DADER’s adversarial alignment approach improves over Ditto zero-shot on three of four evaluated pairs (P1: +0.176, P2: +0.050, P4: +0.064) but degrades on P3 (−0.111), consistent with Tu et al. [5]’s observation that adversarial alignment’s zero-label protection is pair-dependent.